Tokens: How Language Becomes Computation

FRIDAY AI Education — Episode 012 · Printable Companion Paper

Abstract

A language model never sees your words. It sees numbers. Between the text you type and the math the model runs sits a translation layer called **tokenization** — the process of chopping language into small units, mapping each to an integer, and feeding that stream into the model. This step is unglamorous, deterministic, and almost completely invisible. It is also where a surprising amount of your cost, your context limits, and your model's behavior is quietly decided.

This paper explains what tokens are, how text becomes them, why the same information can cost twice as much depending on how it's written, and why the *format* of the context you provide changes the quality of what comes back. The goal is not trivia. It is to give you a mechanical, accurate mental model so that when you design knowledge systems — making a body of public work searchable, citable, and operationally useful — you are reasoning about the machine as it actually is, not as the marketing describes it.

1. Why this is the right place to start

It is tempting to begin the study of AI with the dramatic parts: reasoning, agents, emergent behavior. But every one of those phenomena runs on top of a humble substrate, and if you don't understand the substrate, the dramatic parts will surprise you in expensive ways.

Here is the practical version of the problem. You want to take someone's public body of work — talks, essays, interviews, podcast transcripts — and turn it into something an AI system can search, quote accurately, and act on. To do that you will feed documents into models. You will pay per unit of text. You will hit hard ceilings on how much text fits at once. You will notice that the *same* document, formatted two different ways, produces noticeably different answers.

None of those facts make sense until you understand tokens. Cost is measured in tokens. The context ceiling is measured in tokens. And the model's sensitivity to formatting is a direct consequence of how tokenization and the model's input structure work. So this is not a detour before the interesting material. It is the floor the interesting material stands on.

A clarifying frame for the whole FRIDAY project: the language model is a **reasoning and generation engine with limits**. It is not the system. The system is context plus memory plus tools plus permissions plus evaluation plus audit plus a human approval gate. Tokens are the unit in which the engine consumes its fuel. Knowing the fuel economy is step one.

2. The core concept, stated plainly

A **token** is a chunk of text that the model treats as a single unit. It is usually smaller than a word and larger than a letter. The phrase "language model" might become three tokens; the word "tokenization" might become two or three; a common word like "the" is a single token.

Tokenization is the procedure that converts a string of text into a sequence of these chunks, and then into a sequence of integers — because the model is a mathematical function that operates on numbers, not letters. Every token in the model's known universe has an ID. "Tokenizing" your prompt means looking up each chunk and replacing it with its number.

The **vocabulary** is the fixed, finite list of all tokens the model knows — typically somewhere in the range of 50,000 to 200,000 entries depending on the model. Think of it as the model's complete alphabet, except the "letters" are word-fragments. Everything you ever send the model must be expressed using only the chunks in this list. If a word isn't in the vocabulary as a single token, it gets broken into smaller pieces that are.

That's the whole idea. Text in, chunks out, chunks become numbers, numbers go into the model. The subtlety is entirely in *how* the chunking is done — and that is where the consequences live.

3. The mechanism underneath

3.1 Why not just use words, or letters?

Two obvious schemes both fail.

One token per word seems natural, but it breaks immediately. Language has a near-infinite supply of words — names, typos, technical terms, compounds, new coinages, other languages. A word-level vocabulary would either be impossibly large or would constantly hit words it has never seen and cannot represent at all.

One token per character never hits an unknown input — there are only so many characters — but it makes every sequence enormously long. "transformer" becomes eleven units instead of one or two. The model would spend its limited capacity tracking individual letters instead of meaning, and your costs and context usage would balloon.

The working answer sits between them: **subword tokenization**. Keep common words and fragments as single tokens; break rare words into smaller, reusable pieces. This is the compromise nearly every modern model uses.

3.2 How the pieces get chosen

The dominant family of methods is **Byte-Pair Encoding (BPE)** and its relatives. The intuition is simple and worth internalizing because it explains most of the weird behavior you'll observe.

Before training the model, the tokenizer is *built* by scanning a giant pile of text and learning which character sequences occur together most often. It starts with individual characters and repeatedly merges the most frequent adjacent pair into a new single token. Do this thousands of times and you arrive at a vocabulary where:

- Frequent words ("people," "company," "the") survive as **whole single tokens**.
- Less frequent words get split into a few **common fragments**.
- Genuinely rare strings get broken down toward individual characters or bytes.

This produces a fallback guarantee: because the very bottom of the vocabulary is individual bytes, the tokenizer can *always* represent any input, including emoji, code, and languages it barely saw in training. It just spends more tokens doing so.

The critical consequence: **token count tracks familiarity, not meaning.** Common English prose is cheap — often near four characters per token, roughly three-quarters of a token per word. A dense table of part numbers, a wall of JSON, a transcript full of names and timestamps, or text in a less-represented language can be two to four times more expensive *per unit of actual information*, because the tokenizer has to fall back to smaller pieces.

3.3 What the model actually receives

Once tokenized, your text is a flat list of integers — say, `[791, 4221, 1646, ...]`. The model converts each integer into a vector (an **embedding**, the subject of a later episode), and processes the whole sequence at once. When it generates a reply, it produces tokens one at a time, each appended to the sequence and fed back in to produce the next. The text you read is those output tokens converted back into characters.

Two facts fall directly out of this design, and both matter operationally:

1. **The model has no special access to "words" or "documents."** It sees one undifferentiated stream of tokens. Your prompt, the retrieved source, the system instructions, and the conversation history are all the same kind of thing to it: tokens in a sequence. The structure you think exists is only as real as the structure you encode into that stream.

2. **Position in the stream carries meaning.** Because tokens are processed as an ordered sequence, *where* something sits — early or late, before or after the question, inside a clear delimiter or buried in a paragraph — affects how the model uses it. This is why formatting is not cosmetic.

4. Context length: the hard ceiling

The **context length** (or context window) is the maximum number of tokens a model can hold in mind at once — input and output combined. It is a fixed property of the model. Today's windows range from a few thousand tokens to hundreds of thousands or more, but every model has a wall, and the wall is measured in tokens, not pages or files.

This produces the most common rude surprise in applied AI work: you paste a long PDF and the system refuses it, or silently truncates it, or — worst of all — accepts it but answers as though it only read the first third.

A few honest realities about the window:

- **Pages are a lie; tokens are the truth.** A "20-page PDF" tells you almost nothing. A clean prose essay and a 20-page financial filing dense with tables and figures can differ by 3–4× in token count. Always estimate in tokens. A rough rule for ordinary English: tokens ≈ words ÷ 0.75, or characters ÷ 4. For tables, code, and transcripts, assume worse and verify.
- **The whole conversation competes for the same budget.** In a multi-turn system, the instructions, the retrieved sources, the prior exchanges, and the space reserved for the answer all draw from one pool. A large source pack can crowd out the room the model needs to actually respond.
- **A bigger window is not a substitute for retrieval.** Even when text technically fits, models attend unevenly across a very long context — material in the middle is often used less reliably than material at the edges. Stuffing everything in is not the same as the model *using* everything well.

That last point is the bridge to the FRIDAY design pattern. The disciplined alternative to "put the whole corpus in the window" is **retrieval-augmented generation (RAG)** — *retrieval augmented generation*, the practice of searching your knowledge base for the few most relevant passages and inserting only those into the context. You will study RAG properly in a later episode. For now, hold one idea: the context window is scarce, expensive real estate, and the

job of a good knowledge system is to put the *right* tokens in it, not the *most* tokens.

5. Cost: you are billed by the token

Every commercial model prices usage in tokens, and almost always charges **input tokens and output tokens at different rates**, with output typically the more expensive of the two. This has direct implications for anyone building a system that runs at volume rather than as a one-off chat.

Three consequences worth designing around:

Format efficiency is real money. The same information, expressed verbosely, costs more every single time it passes through a model. Across thousands of runs, the difference between a tidy 800-token source passage and a sloppy 2,000-token one is not a rounding error; it is the unit economics of your product.

Repeated context is repeated cost. If your system re-sends the same large instructions or the same source pack on every request, you pay for it on every request. This is why caching, summarization, and retrieval (sending only what's needed) are cost levers, not just performance ones.

Compression is a first-class technique. When source material is too large or too expensive to send raw, you can pre-process it: summarize, extract structured fields, strip boilerplate, or distill a long transcript into its claims. This trades a one-time processing cost for repeated savings — and, done carefully, often *improves* answer quality by removing noise. The risk, which you must respect, is that compression is lossy: every summary discards something, and if you discard the wrong thing, the model now reasons confidently over an incomplete record. Compress deliberately, keep the original, and be able to cite back to it.

A compact reference table

Concept	What it is	Unit / measure	Why it matters operationally
Token	A chunk of text treated as one unit	~¾ word for English prose	The atom of everything below
Tokenization	Text → chunks → integers	Deterministic procedure	Decides token count for any input
Vocabulary	The fixed list of known tokens	~50k–200k entries	Anything unknown is split into smaller pieces
Context length	Max tokens held at once (in + out)	Fixed per model	Hard ceiling; PDFs measured here, not in pages
Cost	Charged per token	Input rate + output rate	Verbose formatting = recurring expense
Compression	Pre-shrinking text before sending	Tokens saved per run	Cheaper and often cleaner — but lossy

6. Why formatting the context changes behavior

This is the section to read twice, because it is the least intuitive and the most useful.

Since the model sees only a flat sequence of tokens, it relies on **patterns in that sequence** to know what role each part plays. It learned during training that certain structures — clear headings, labeled sections, consistent delimiters, question placed after the relevant material — tend to precede good, well-organized responses. When you format your context to match those structures, you are effectively steering the model toward the behavior you want. When you hand it an undifferentiated blob, you force it to guess where the instructions end and the data begins.

Three concrete, repeatable wins:

Label your roles. Mark what is an instruction, what is source material, and what is the question. A passage introduced with `SOURCE (Q3 earnings call, 2026):` followed by the text, and then a clearly separated `QUESTION:`, is dramatically easier for the model to use correctly than the same content run together. You are giving structure to a stream that has none by default.

Put the question in the right place relative to the evidence. Because order matters, stating the task and then providing the evidence — or providing evidence and then a crisp instruction about what to do with it — beats burying the ask in the middle of a long dump.

Make boundaries explicit. Delimiters (headings, fenced blocks, consistent separators) reduce the chance the model bleeds one section into another — for instance, treating a quote inside a source as if it were your instruction. This is also a quiet security property: clear boundaries make it harder for text inside a document to be misread as a command. You'll meet this properly under prompt-injection later; for now, note that format is partly a safety control.

The takeaway: **the model's answer is a function of the token stream, and you control the token stream.** Formatting is not decoration. It is the primary, cheapest lever you have over output quality, and it costs you nothing but discipline.

7. Bringing it together: the FRIDAY knowledge-base case

Consider the first practical wedge: taking a person's or company's public body of work and making it searchable, reusable, cited, and operationally useful. Walk it through everything above.

Source packs. A "source pack" is the bundle of material you assemble to answer a question. Tokens tell you it must be *curated*, not dumped. The window is finite and every token is billed, so the pack should contain the few most relevant passages, clearly labeled with their origin so the system can cite them. The quality of the pack — relevance, structure, provenance — usually matters more than its size.

Long PDFs. These are the canonical trap. A founder's 60-page deck or a 40-page report will rarely fit cleanly, will tokenize expensively because of tables and figures, and will be used unevenly even if it fits. The correct handling is to break it into passages, store them, retrieve only what a given question needs, and keep a pointer back to the page so answers remain citable. Pages are for humans; tokens are for the machine; provenance is for trust.

Telegram transcripts. Chat logs are deceptively costly. Names, handles, timestamps, emoji, and fragmentary phrasing all tokenize poorly, so a transcript that looks short can consume a large share of the window. They also have weak inherent structure, which makes formatting and pre-processing especially valuable: cleaning, attributing speakers consistently, and compressing runs of low-content chatter into summarized claims can cut cost sharply and improve what the model extracts — provided you keep the raw log and can point back to it.

Why formatting context matters here, specifically. Your knowledge base will feed the model retrieved fragments from many sources at once. Without clear labels and boundaries, the model cannot reliably tell one source from another, attribute a claim correctly, or distinguish a quotation inside a document from an instruction to itself.

Disciplined formatting is what makes citations trustworthy and behavior predictable — which is the entire point of building a knowledge base someone would rely on.

The shape of the system, then, is not "a big model with a big window." It is GBrain as the memory and knowledge substrate, a retrieval step that selects the right tokens, deliberate formatting that tells the model how to use them, and — because none of this is self-policing — evals to measure whether answers are actually grounded, audit to trace what was used, and a human approval gate before anything acts on the output. The model is the engine. Tokens are how you feed it well.

8. The main limitation and failure mode

The honest summary of tokenization's downside: **the model's view of your text is shaped by the tokenizer, and the tokenizer has blind spots.**

The clearest failure mode is the **counterintuitive cost and capacity of "small-looking" inputs.** A short table, a code snippet, a non-English passage, or a chat transcript can consume far more tokens than its length on screen suggests, because the tokenizer falls back to smaller pieces for anything outside its comfort zone of common prose. Teams routinely under-budget for exactly this and are surprised by both the bill and the truncation.

A subtler failure mode is **silent truncation and uneven attention.** When input exceeds the window, something gets dropped — often without a loud error — and the model answers as if it read everything. Even within the window, material buried in the middle of a long context may be underused. Both produce confident, fluent answers built on an incomplete reading. The defense is operational, not magical: estimate tokens before you send, retrieve rather than dump, place critical material where it will be used, and *test* — run evals that check whether the answer is actually grounded in the source, rather than trusting that it must be because the source was "in there somewhere."

And the standing caveat that applies to all of this: tokenization is a mechanical convenience, not comprehension. The model is manipulating familiar sequences of chunks, not reading the way you do. Treat fluent output as a draft to verify, never as truth — especially once a system is allowed to act on what it produces.

9. Vocabulary

- **Token** — A chunk of text treated as a single unit by the model; usually a word-fragment, roughly three-quarters of a word for English prose.
- **Tokenization** — The deterministic process of converting text into tokens and then into integer IDs the model can process.
- **Vocabulary** — The fixed, finite list of all tokens a model knows; everything sent to the model is expressed using only these.
- **Subword tokenization / Byte-Pair Encoding (BPE)** — The dominant method: keep frequent words whole, split rare ones into reusable fragments, with single bytes as the universal fallback.
- **Context length / context window** — The maximum number of tokens (input plus output) a model can process at once; a fixed hard ceiling.
- **Prompt formatting** — Deliberately structuring the token stream (labels, delimiters, ordering) to steer model behavior and output quality.
- **Cost (token-based pricing)** — Billing measured in input and output tokens, typically at different rates, recurring on every request.

- **Compression** — Pre-processing text (summarizing, extracting, stripping boilerplate) to reduce token count before sending; cheaper and often cleaner, but lossy.
- **Source pack** — A curated bundle of relevant, labeled, provenance-tagged passages assembled to answer a question.
- **RAG (retrieval-augmented generation)** — Searching a knowledge base for the most relevant passages and inserting only those into the context, instead of sending everything.
- **Provenance** — The recorded origin of a piece of text, enabling citation and verification back to the source.

10. Reading guide

Two short, hands-on anchors. Neither is a course; both are worth twenty minutes with your own material.

OpenAI Tokenizer — *https://platform.openai.com/tokenizer*

An interactive tool that shows exactly how a piece of text is split into tokens and counts them. Don't just read about tokenization — *do* it. Paste in three things and compare the token counts: (1) a paragraph of clean English prose, (2) a small table or some JSON, and (3) a snippet of a chat transcript with names and timestamps. Watch the cost-per-information difference appear in front of you. This single exercise will make the abstract claims in Sections 3 and 5 concrete and permanent.

Hugging Face — Tokenizer Summary — *https://huggingface.co/docs/transformers/tokenizer_summary*

A clear technical overview of the main tokenization schemes (BPE and its relatives) and why subword methods won. Read it for the *why* behind Section 3.2 — particularly how the vocabulary is learned from frequency before the model is ever trained. Skim the algorithm details if they slow you down; the conceptual through-line is what matters. Read it after you've played with the tokenizer, so you have concrete observations to attach the theory to.

11. Walking questions

Carry these on the walk. They are for thinking, not for homework.

1. Which part of tokenization could you explain clearly to a smart non-technical partner right now — and where would you start hand-waving?
2. Where in a real FRIDAY or GBrain workflow does the token budget actually bind first: cost, the context ceiling, or formatting quality?
3. Think of one document you'd genuinely want in a knowledge base. Would it fit a context window? How would you estimate that without guessing?
4. What is one small, bounded workflow — a single document type, a single question — where you could test retrieval-and-format discipline safely before trusting it with anything that matters?

One-sentence takeaway

A model reasons over a stream of tokens it didn't choose, so the leverage in any AI system lives in deciding *which* tokens reach it and *how* they're arranged — that, not the model itself, is the part you control.